迷你搜索引擎

倒排索引的构建、查询与阈值分析 • 数据结构课程项目 8-1

C++17　主语料：莎士比亚全集（195 篇）　副语料：IETF RFC ×3

2026 年 10 月

概览

1. **设计流程**
2. **测试与结果**
3. **Bonus 的结果与分析**
4. **项目评价**

三组数字

语料规模（主语料）	**195 篇**
正文词元	**866,626**
正式索引词项	**14,937**
正式索引体积	**3.69 MB**
阈值实验测量	**201 条**
CTest 断言	**85,155**

四项课程任务全部完成并经独立对拍；Bonus 的 **50 万文件轴实测通过**，**4 亿词轴**在实测锚点上外推，两条轴在页面上始终标清 “实测 / 外推”。

报告摘要、§3、§4；REVIEW.md 数字审计

任务与语料

课程任务：4 项 + Bonus

1. 词频统计与**停用词**界线
2. 带**Porter 词干化**的倒排索引（不含停用词）
3. 单词 / 短语（及布尔）查询，返回文档 ID
4. 查询阈值实验：`max-df`、`min-df`、`min-match`、`top-x`

Bonus：50 万文件 + 4 亿不同词

语料	主语料：莎士比亚全集	副语料：IETF RFC
文档数	195（37 剧 / 154 十四行诗 / 4 长诗）	3
词数（空白切分）	930,911	81,848
正文词元（默认配置）	866,626	75,250
语料字节	4,919,371 B	569,472 B

语料从镜像的 1,211 个 HTML 按清单抽取；同一套二进制不改代码跑两份语料，Python 全词表对拍一致。

tasks F1/F3；verify F10；报告摘要、§4.1

构建流水线



构建侧

- 词典可插拔：伸展树 / 哈希 / `std::map`
- 索引可写 v1（原始 u32）或 v2（差分 + varint）
- 大语料走 SPIMI 式块式构建

查询侧

- 单词 / 短语 / 布尔，输出 JSON
- 四类阈值：`max-df`、`min-df`、`min-match`、`top-x`
- 冷查询走磁盘 B+ 树附属词典

语料清单 + 纯文本层是**唯一接口**：换语料只改 producer，不改索引与查询代码。

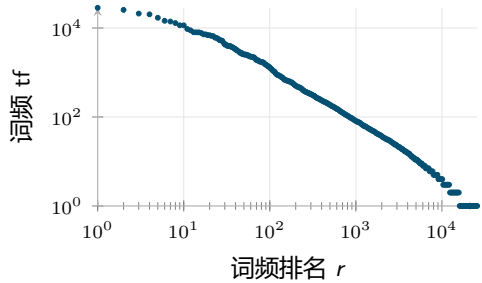
报告 §1–§2；tab:coverage

词频统计：Zipf 头部集中

词表结构（默认配置）

不同词形	25,705
仅出现一次（hapax）	9,753
top-10 词元占比	20.41%
top-100 词元占比	54.30%
top-1,000 词元占比	80.26%

- tf / df 双指标并列（the：tf 28,436 / df 177）
- 全量 CSV 由独立 Python 实现**逐字节对拍**



主语料 tf–排名双对数图：近似 $1/r$ 衰减（图 1）

观察

极少数词覆盖大量词元，这正是停用词值得单独处理的统计原因；同时 hapax 占词表 **37.9%**，是词干化能大幅合并词表的空间所在。

tasks F6–F8；wordcount-default.csv；报告 §2.4

停用词：从人工候选带到统计规则

第一阶段：tf/df 候选带扫描 + 人工定界

按候选带 $4 \to 9 \to 14 \to 16 \to 21 \to 31 \to 48$ 词逐步放开，观察 tf/df 曲线拐点，人工定界得到 **39 词**。

第二阶段："宽、重、平" 三特征规则（纯统计，无人为挑选）

宽：$\mathrm{df} \geq 0.40\times$ 文档数

重：tf 占比 ≥ 0.001

平：长度归一化速率熵 ≥ 0.93

结果：同样 39 词，与人工表重叠 35

效果与边界

39 个词只占词表的 **0.15%**，却覆盖 **35.74%** 的正文词元（移除 309,760 个）。速率熵度量 "分布是否平坦"，是三特征里区分功能词与内容词的决定性一条。

tasks F9/F11–F14；报告 §2.5、§3.5

停用词没有万能表：RFC 交叉验证

同一套三特征规则 $(0.40, 0.001, 0.93)$	莎士比亚	IETF RFC
推导出的停用词数	39	**58**
与莎士比亚 39 词表的重叠	—	仅 26 词
索引体积（原始 → 正式配置）	5,598,793 → 3,685,655 B	510,181 → 308,164 B

- RFC 2119 关键词 `must` / `may` / `should`：df 比值均为 1（足够 “宽”“重”），但速率熵 **0.809 / 0.899 / 0.885** < 0.93（不够 “平”）→ **正确排除**
- `server`、`page`、`rfc` 速率熵接近 1 → 收入推导表（`[Page N]` 样板属 producer 侧噪声，非引擎问题）
- 端到端：RFC 上原始配置 75,250 词元 → 正式配置 45,370 词元；`brute_force.py` 全词表对拍一致

结论

同一套阈值在不同语料上给出不同的停用词表——停用词是**语料相关**的，规则可以复用，词表必须重推。

verify F10；报告 §4.1、表 tab:rfc

Porter 词干化与四种索引配置

自实现 Porter 词干化

25,705 词形 → **14,947 词干**（**−41.85%**），hapax 9,753 → 4,843；词干化把 “同源词形” 折叠为一个词项，直接缩小词表与倒排记录。

配置（flags）	词项数	倒排记录	索引字节
raw（无过滤、无词干）	25,705	152,650	5,598,793
stop（停用词）	25,666	147,909	4,320,622
stem（词干化）	14,947	121,903	4,958,598
正式（stopwords + stemmed）	**14,937**	**117,704**	**3,685,655**

结论

停用词与词干化分别压缩 “词元流长度” 与 “词表大小”，两者叠加使正式配置最小；四种配置全部通过独立 Python 实现的对拍（词频 CSV 逐字节、倒排记录全项一致）。

tasks F18；报告表 tab:index-configs

索引格式：v1 布局与 v2 压缩

v1：原始 u32 布局

- 文件头 + 文档表 + 有序词表 + 倒排记录区
- 倒排记录：docID、tf、位置，各 4 字节

v2：只改倒排记录区

- docID 与位置存**差分**
- tf 与差分值用**字节对齐变长码**（varint）

语料	第一版字节	第二版字节	第二版/第一版	压缩构建峰值内存
主语料（莎士比亚）	3,685,655	**1,725,694**	**46.82%**	0.19 s（内建）
500k 合成语料	427,603,247	284,560,009	66.55%	1.46 GiB（v1 为 1.01）

成本权衡

- 主语料倒排记录区占 86.0%、段内节省 61.8% → 整体节省 53.18%
- 500k 词项表占比高（倒排区仅 56.1%），段内节省 59.6% 与主语料接近
- 压缩不是全维度占优：构建峰值内存 1.01 → **1.46 GiB**；查询输出与 v1 完全一致

tasks F21；verify F13/F14；报告 §3.4

三种词典：构建过程不同，产物相同

词典（正式配置）	比较 / 旋转	额外事件	构建耗时
伸展树（课程要求，自实现）	6,392,724 / 5,850,795	—	0.19 s
哈希表	688,494	11 次 rehash	**0.09 s**
`std::map`（基线）	9,435,694	—	0.14 s

结论

哈希最快；伸展树最慢，但中序遍历**直接产出有序词表**，且满足课程对自实现数据结构的要求。三种词典写出的索引**逐字节相同**（SHA-256 一致）：词典只改变构建过程，不改变持久化结果。

提示：计时受并行负载影响（报告已声明 ±20% 量级参考），结构性数字（字节数、SHA）不受影响。

选型建议：自实现伸展树满足课程要求且中序有序；若只追求构建速度，哈希约快 2 倍——两者写出的索引字节完全相同，因此选型只影响构建阶段。

tasks F20；verify F5；报告 §4.3

磁盘 B+ 树附属词典与 SPIMI 块式构建

B+ 树 sidecar（查询侧）

词项 → 倒排记录（偏移，块长）的磁盘映射：

- 文件仅 **237,568 B**（索引的 6.45%）
- 冷查询只读 **8,308 B** = 全量加载的 **0.225%**
- 与全量加载逐词一致（双路径校验）

SPIMI 块式构建（构建侧）

内存预算 4,096 活词：

- 17 个临时块、峰值活词 **5,129**
- 临时块共 4,166,556 B（用后即删）
- 归并产物与内存构建**逐字节一致**

为什么是这两个部件

两者合起来去掉 “构建/查询必须把全词表放进内存” 的假设：查询侧把词典移到磁盘，构建侧把内存压到 O(预算) 与词表规模无关——这是 Bonus 两条规模轴的工程基础。

tasks F22；verify F23；报告 §2.7、§2.10

查询设计与样例

支持单词、短语（词元流上整段匹配）与布尔（AND 语义，按命中词数排序）；输出 JSON，`--pretty` 可读。

查询	行为	结果
`osric`	稀有词，单文档角色	1 篇
`love`	高频词（top-3 tf 195 / 165 / 142）	138 篇
`"king richard"`	短语整段匹配	7 篇（top-3 tf 193 / 191 / 140）
`"my lord"`	`my` 为停用词 → 只查 `lord`	42 篇
`king queen`	布尔：按命中词数 + tf 排序	33 篇双命中 / 43 篇单命中

短语语义的边界

位置定义在**过滤后**的词元流上：`"my lord"` 退化为 `lord`，全由停用词组成的短语返回空——这是设计选择，不是实现缺陷（见第 4 章局限）。

排序仅用命中词数与 tf 总和，**无 idf 与长度归一化**，长剧更易靠前。

tasks F27–F33；报告 §3.8

阈值实验：阈值改变的是查询语义

15 条查询 × 四类阈值 = **201 条测量**（中位 19.2 ms，总计 4.05 s）；用 Jaccard 系数对比阈值前后的结果集。

反直觉但可解释的三个现象

1. **收紧 ≠ 变少**：`sweet sword moon` 加 `--min-df 50` 后 `sword` 等词被裁，布尔查询退化为单词 `moon`，结果 **30 → 90 篇**（Jaccard 0.333）
2. **短语不受词侧阈值影响**：`"king richard"` 全档 7 → 7（Jaccard 1.00）——短语必须整段匹配，裁剪查询词只会让短语失效或不变
3. **停用词查询恒为空**：`the` / `of` 全档 0 → 0——过滤索引的设计行为，非 bug

`--top-x` 才是真正的**结果截断**：love 的 top-5 / 10 / 20 对应 Jaccard 0.036 / 0.073 / 0.145。

`--min-match` 改变**布尔条件**（AND 要求命中词数）：1 / 2 / 3 对应 93 / 39 / 30 篇。

一句话总结：`max-df/min-df` 是查询词筛选器（偏精度），`min-match` 改布尔语义，`top-x` 才截断结果——四者作用在不同阶段，所以 “阈值收紧” 不能简单等同于 “结果变少”。

tasks F36–F39；threshold_matrix.csv；报告 §3.9

Bonus 1：磁盘 B+ 树词典的规模实测

4,908,747 词项的内存索引峰值 2.00 GiB → 线性外推 4 亿词 ≈ **160 GiB**（外推），单机放不下：词典必须上磁盘。

键数	树高	总页数	文件字节	字节/键	页读/查询
100,000	2	369	1,515,520	15.1552	2
1,000,000	3	3,688	15,110,144	15.1101	3
10,000,000	3	36,865	151,003,136	15.1003	3
30,000,000	3	110,593	452,993,024	15.0998	3
50,000,000	4	184,322	754,987,008	15.0997	4

外推与边界

4 KiB 页、批量装载、每档 1,000 次随机查询全部校验；体积近似**严格线性**（15.10 B/键）。外推 4 亿键 ≈ **6.04 GB**、树高 **4**、单查询读 4 页 = **16 KiB**（树高与页读已在 5,000 万键**实测**到 4，外推的只有文件字节与页数）。

verify F11；bptree-bench.csv；报告表 tab:bptree-scale

Bonus 2：500,000 文档合成语料（实测）

合成语料：500,000 个文件、20,000,000 词元、4,908,747 个不同词、19,999,929 条倒排记录。

构建方式	耗时	峰值内存	产物
块式 v1（4,855 块）	扫描 5.47 + 归并 125.08 s	**1.01 GiB**	427,603,247 B
块式 v2（差分压缩）	扫描 5.42 + 归并 123.31 s	1.46 GiB	284,560,009 B
内存构建 v1	总 76.75 s（内建 44.93 s）	2.00 GiB	427,603,247 B

结论

块式与内存构建的 v1 索引**逐字节相同**（SHA-256 一致）：内存 O(预算) 与词表规模无关，在 50 万文件规模**实测成立**。代价是约 **2.2 倍墙钟**——归并现为 $O(T \cdot R)$ 线性扫游标，换成堆可降到 $O(T \log R)$。

verify F12–F15；报告表 tab:500k

正确性：可复现的验证链

- CTest **18/18** 通过：8 个 Catch2 可执行文件、46 个用例、**85,155 条断言**（B+ 树占 99.3%）
- 独立 Python 实现**对拍**（不经任何项目代码路径）：
  - 四种配置词频 CSV 逐字节一致（1,677,253 / 1,674,689 / 958,633 / 957,871 B）
  - 正式 / 原始 / 压缩索引的全词表 df、tf、docID、位置全项一致
  - 短语对拍：305 条采样、303 条检查、**0 不一致**（21.20 s）
  - 磁盘词典双路径：B+ 树查询与全量加载逐词一致
- 讲义四文档黄金样例逐项吻合；块式 / 内存 / 三词典产物 SHA-256 相同

项目的最大价值

不是某个墙钟数字，而是这条验证链：同一文本层能被独立实现重建，三种实现写出同一字节流，“看起来能跑”的错误在提交前就被暴露。

verify F1–F9；tasks F15；报告 §3.1–§3.4、§4.4

局限与结论

主要局限

- 排序无 idf / BM25，长剧占优
- 短语匹配在过滤后词元流上（`"my lord"` 退化）
- 切词仅 ASCII；构建与查询单线程
- 无增量索引；4 亿词未端到端实测

下一步（按收益排序）

1. 粒度实验（scene 级 / 合并短诗）
2. BM25 / TF-IDF 与相关性评价
3. SPIMI 归并输出直接写 B+ 树，跑一次真正 500k/400M 端到端

结论

四项课程任务全部完成并经独立对拍；**50 万文件轴实测通过**，**4 亿词轴**给出实测锚点上的外推（6.04 GB / 树高 4 / 4 页读 = 16 KiB），两条轴在全文都明确区分 “实测” 与 “外推”。

完整停用词表见附录 A / 附录 B。

报告 §3.5、§4.2；REVIEW.md 数字审计

附录 A：主语料的停用词（人工 39 / 统计 39）

人工定界（39 词，data/stopwords.txt）

the, and, i, to, of, a, my, that, in, is, not, me, it, for, with, be, his, this, but, have, as, so, what, by, no, all, do, if, are, on, from, or, which, more, then, their, when, than, yet

三特征统计推导（39 词，df≥0.40、tf_share≥0.001、熵 ≥0.93）

the, and, i, to, of, a, my, that, in, is, not, me, it, for, with, be, his, this, but, have, as, **thou**, so, what, by, no, all, do, if, are, **thee**, on, from, **o**, which, then, when, **hath**, yet

两份表的差异：重叠 35 / 39

推导**多出** thou、thee、o、hath（古英语功能词——“分布平坦” 能自动识别它们）；人工表**多出** more、their、than、or（比较与选择连词，速率熵略低于 0.93，被判为 “不够平”）。

tasks F9/F11–F14；data/stopwords.txt；derived-df0.40-er0.93.txt；logs/tasks/stopwords-list.log

附录 B：副语料 RFC 的停用词（58 词，纯统计推导）

the	a	to	of	and
in	is	that	for	be
rfc	or	not	**server**	as
this	are	an	by	it
with	if	**data**	from	**page**
information	on	used	when	any
standards	use	can	other	one
e	will	some	have	all
protocol	no	only	case	its
more	**list**	**error**	**time**	received
each	does	same	they	g
transfer	however	multiple		

读法

同一套规则 $(0.40,\ 0.001,\ 0.93)$ 得到 58 词：**加粗**的 server / page / rfc / data / information / protocol / error / time / list / transfer 是典型**领域内容词**，而 RFC 2119 的 must / may / should（熵 0.809 / 0.899 / 0.885 $<$ 0.93）**不在表中**。

谢谢！问题与讨论欢迎提出。

verify F10；work/verify/rfc-stopwords.txt（e / g 来自 “e.g.” 切分）；报告 §4.1